NATIONAL TEACHERS COLLEGE
629 J. Nepomuceno Street, Quiapo, Manila, Philippines
SCHOOL OF ARTS, SCIENCES, AND TECHNOLOGY
Bachelor of Science in Information Technology
Academic Term 2025-2026

Laboratory Examination
Weeks 7–11 Integrated Dart Console Application

Presented by:

Delave, Jr., Jonathan, P.

3.1 BSIT

Instructor:

Mr. Edward James V. Grageda

Mobile Development w/Lab

November 10, 2026

PART 1 – Dart Fundamentals

```dart
void main() {
  print('PART 1: DART FUNDAMENTALS');

  int ticketId = 2501;
  String studentName = 'Jonathan';
  String serviceType = 'Printer Repair';
  double repairCost = 1200.00;
  bool isPriority = true;

  double totalAmount = repairCost + 300.00;
  bool isHighCost = totalAmount > 1000.00;

  print('Ticket ID: $ticketId');
  print('Student Name: $studentName');
  print('Service Type: $serviceType');
  print('Repair Cost: PHP $repairCost');
  print('Priority: $isPriority');
  print('Total Amount: PHP $totalAmount');
  print('High Cost: $isHighCost');

  print('\nSecond Ticket:');
  print('Ticket ID: 2502');
  print('Student Name: Meg Madrid');
  print('Service Type: Computer Cleaning');
  print('Repair Cost: PHP 450.00');
  print('Priority: false');
}
```

Output:

```
PART 1: DART FUNDAMENTALS
Ticket ID: 2501
Student Name: Jonathan
Service Type: Printer Repair
Repair Cost: PHP 1200
Priority: true
Total Amount: PHP 1500
High Cost: true

Second Ticket:
Ticket ID: 2502
Student Name: Meg Madrid
Service Type: Computer Cleaning
Repair Cost: PHP 450.00
Priority: false
```

PART 2 – Program Logic and Error Handling

```dart
String checkPriority(bool priority) {
  if (priority) {
    return 'High Priority';
  } else {
    return 'Regular Priority';
  }
}

void main() {
  print('PART 2: PROGRAM LOGIC');

  List<bool> tickets = [false, true, false, true];

  for (int i = 0; i < tickets.length; i++) {
    String status = checkPriority(tickets[i]);

    print('Ticket ${i + 1}: $status');
  }

  print('\nERROR HANDLING');

  try {
    int ticketNumber = int.parse('XYZ');
    print(ticketNumber);
  } catch (e) {
    print('Invalid ticket number. Please enter numbers only.');
  }
}
```

Output:

```
PART 2: PROGRAM LOGIC
Ticket 1: Regular Priority
Ticket 2: High Priority
Ticket 3: Regular Priority
Ticket 4: High Priority

ERROR HANDLING
Invalid ticket number. Please enter numbers only.
```

PART 3 – OOP: Classes, Constructors and Encapsulation

```dart
class ServiceTicket {
  int ticketId;
  String studentName;
  String problem;
  double cost;

  String _progress = 'Waiting';

  ServiceTicket(
    this.ticketId,
    this.studentName,
    this.problem,
    this.cost,
  );

  String get progress => _progress;

  void changeProgress(String newProgress) {
    _progress = newProgress;
  }

  void showDetails() {
    print('Ticket ID: $ticketId');
    print('Student: $studentName');
    print('Problem: $problem');
    print('Cost: PHP $cost');
    print('Progress: $progress');
  }
}

void main() {
  print('PART 3: OBJECT-ORIENTED PROGRAMMING');

  ServiceTicket ticket1 =
      ServiceTicket(3001, 'Jonathan Delave', 'Keyboard Problem', 500.00);

  ServiceTicket ticket2 =
      ServiceTicket(3002, 'Meg Madrid', 'Monitor Repair', 1800.00);

  ServiceTicket ticket3 =
      ServiceTicket(3003, 'Abram Madrid', 'Slow Computer', 750.00);

  ticket1.changeProgress('In Progress');
  ticket2.changeProgress('Completed');

  List<ServiceTicket> ticketList = [
    ticket1,
    ticket2,
    ticket3
  ];

  for (var ticket in ticketList) {
    ticket.showDetails();
```

```dart
    print('');
  }

  Map<String, String> schoolInfo = {
    'Department': 'Computer Laboratory',
    'Service': 'Technical Support',
    'Location': 'Room 201'
  };

  print('SCHOOL INFORMATION');

  schoolInfo.forEach((key, value) {
    print('$key: $value');
  });
}
```

Output:

```
PART 3: OBJECT-ORIENTED PROGRAMMING
Ticket ID: 3001
Student: Jonathan Delave
Problem: Keyboard Problem
Cost: PHP 500
Progress: In Progress

Ticket ID: 3002
Student: Meg Madrid
Problem: Monitor Repair
Cost: PHP 1800
Progress: Completed

Ticket ID: 3003
Student: Abram Madrid
Problem: Slow Computer
Cost: PHP 750
Progress: Waiting

SCHOOL INFORMATION
Department: Computer Laboratory
Service: Technical Support
Location: Room 201
```

PART 4 – OOP: Inheritance, Polymorphism and Reuse

```dart
mixin ActivityLogger {
  void recordActivity(String message) {
    print('[ACTIVITY] $message');
  }
}

abstract class TechnicalService {
  void performService();
}

class ComputerRepair extends TechnicalService
    with ActivityLogger {

  @override
  void performService() {
    print('Service: Fixing a computer problem.');
    recordActivity('Computer repair has been recorded.');
  }
}

class PrinterRepair extends TechnicalService
    with ActivityLogger {

  @override
  void performService() {
    print('Service: Fixing a printer problem.');
    recordActivity('Printer repair has been recorded.');
  }
}

void main() {
```

```dart
void main() {
  print('PART 4: INHERITANCE AND POLYMORPHISM');

  List<TechnicalService> services = [
    ComputerRepair(),
    PrinterRepair()
  ];

  for (var service in services) {
    service.performService();
    print('');
  }
}
```

Output:

```
PART 4: INHERITANCE AND POLYMORPHISM
Service: Fixing a computer problem.
[ACTIVITY] Computer repair has been recorded.

Service: Fixing a printer problem.
[ACTIVITY] Printer repair has been recorded.
```

PART 5 – Asynchronous Programming

```dart
import 'dart:async';

class TicketSaver {
  Future<String> saveTicket() async {
    await Future.delayed(Duration(seconds: 3));

    return 'Technical support ticket saved!';
  }
}

Future<void> main() async {
  print('PART 5: ASYNCHRONOUS PROGRAMMING');

  TicketSaver saver = TicketSaver();

  print('Processing ticket...');

  Timer(Duration(seconds: 1), () {
    print('Timer: Checking ticket information...');
  });

  String message = await saver.saveTicket();

  print(message);
  print('All tasks are completed!');
}
```

Output:

```
PART 5: ASYNCHRONOUS PROGRAMMING
Processing ticket...
Timer: Checking ticket information...
Technical support ticket saved!
All tasks are completed!
```

Explanation

1. I created a service ticket using the name Jonathan. I used int for the ticket ID and String for the student name and service type. I used double for the repair cost and bool for the priority status. I added PHP 300 to calculate the total amount. I used a comparison operator to check if the total amount was above PHP 1,000. The program displays two sample service tickets with their details.

2. I created a function named checkPriority to check the priority of a ticket. The function uses a boolean parameter and returns High Priority or Regular Priority. I used if/else statements to check the ticket status. I used a for loop to process four sample tickets. I also used a try/catch block to handle an invalid number. The program displays a helpful error message when the input is incorrect.

3. I created a class named ServiceTicket to store technical support information. I used a constructor to create three ticket objects. I added a showDetails() method to display the information of each ticket. I used a private field named _progress and a getter to access its value. I also created a method to change the progress of a ticket. I used a List to store three objects and a Map to store school information.

4. I created an abstract class named TechnicalService. I created two child classes named ComputerRepair and PrinterRepair. Both classes override the performService() method. The ComputerRepair class displays a message about fixing a computer. The PrinterRepair class displays a message about fixing a printer. I used an ActivityLogger mixin to share a logging function. I also used polymorphism to call the same method for different service objects.

5. I created a class named TicketSaver to handle ticket saving. I added a saveTicket() method that returns a Future String. I used Future.delayed() to simulate a three-second saving process. I used async and await to wait for the saving process to finish. I also used Timer to display a checking message after one second. The program displays the timer message first, followed by the successful saving message.